

Day 41 – Hibernate (Day 02)

SessionFactory Recap • Entity Class & JPA Annotations • Hibernate Session and Transaction

1. Recap – Configuration and SessionFactory

We start from where we left off. The following code establishes the connection between our Java application and Hibernate using the configuration file.

App.java

```
public class App
{
    public static void main( String[] args )
    {
        Configuration cnfig = new Configuration();
        cnfig.configure("resources/hibernate-cnf.xml");

        SessionFactory factory = cnfig.buildSessionFactory();
        System.out.println(factory);
    }
}
```

What each line means:

This line will:

```
Configuration cnfig = new Configuration();
cnfig.configure("resources/hibernate-cnf.xml");
```

- Read the XML file
- Parse the XML file
- Read all the properties present in hibernate-cnf.xml

Once the configuration is complete, **buildSessionFactory()** creates the **SessionFactory** object.

NOTE: This SessionFactory object is provided by Hibernate itself. That is why we do not create it using the **new** keyword — we obtain it through **cnfig.buildSessionFactory()**.

Now that the connection is established using the code above, let us see how we can perform operations such as creating a table and inserting data.

As we already know, Hibernate manages all of this for us. We are not going to write any SQL query, and we are not going to create any table manually. Hibernate tells us that it will manage all of this with the help of JPA rules for performing these operations.

The first requirement is creating a table so that we can perform some operation on it. Let us see how.

2. Hibernate Entity Class and JPA Annotations

2.1 Requirement

Suppose our requirement is to store employee information in the database. For example, we want to store:

id	name	address	companyName
101	Prashant	Hyderabad	ABC
102	Dhoni	Chennai	XYZ

To store this information, we need a table in the database. At the same time, we need a way to represent this database data inside our Java application. For that, we create a Java class called **Employee**.

2.2 What is an Entity Class?

An Entity Class is a Java class whose objects represent the data that we want to store and manage in the database.

Example

```
public class Employee {  
  
    private int id;  
    private String name;  
    private String address;  
    private String companyName;  
  
}
```

Here, the Employee class represents employee information.

If we create an object:

```
Employee emp = new Employee();
```

that object can represent one employee. For example, the Employee object below:

Employee Object

```
id -> 101  
name -> Prashant  
address -> Hyderabad  
companyName -> ABC
```

represents one employee's data. We can understand the relationship like this:

Java Side	→	Database Side
Employee Class	→	Employee Table

Employee Object	—>	One Row / Record
Object Properties	—>	Table Columns

DEFINITION: Entity Class = A Java class whose objects represent data that is stored and managed in the database.

2.3 @Entity Annotation

If we simply create a Java class:

```
public class Employee {
    ...
}
```

Hibernate does not automatically know that this class should be treated as an entity. So, we use the **@Entity** annotation.

```
@Entity
public class Employee {
    ...
}
```

@Entity tells Hibernate:

HIBERNATE: "Employee is an Entity class. Hibernate should manage its objects as data."

In simple words — **@Entity** tells Hibernate that this Java class represents an entity.

2.4 @Table Annotation

Now suppose our Java class is:

```
@Entity
public class Employee {
    ...
}
```

But in the database, we want the table name to be **employee_details**. We can specify the table name using **@Table**.

```
@Entity
@Table(name = "employee_details")
public class Employee {
    ...
}
```

@Table(name = "employee_details") tells Hibernate:

HIBERNATE: "Map this Employee Entity to the employee_details table."

So the mapping is:

Java	→	Database
Employee Class	→	employee_details Table

Why do we use @Table?

We use @Table when we want to explicitly specify the database table to which the Entity should be mapped. Example:

```
@Table(name = "employee_details")
```

NOTE: @Table is not mandatory in every case. If we don't specify @Table, Hibernate can determine the table name using its own naming rules. But when we want to explicitly specify a table name, we use @Table.

2.5 @Id Annotation

Now consider the following table:

employee_details

id	name	address
101	Prashant	Hyderabad
102	Dhoni	Chennai
103	abc	Mumbai

Suppose we want to uniquely identify Prashant. We can use:

```
id = 101
```

Similarly:

```
101 -> Prashant
102 -> Dhoni
103 -> abc
```

Here, the **id** uniquely identifies each employee. Therefore, id is the Primary Key.

In the Entity class, we tell Hibernate that id is the identifier by using @Id.

```
@Id
private int id;
```

@Id tells Hibernate:

HIBERNATE: "This field uniquely identifies the Entity."

In database terminology, this normally corresponds to the Primary Key.

2.6 Complete Entity Class

Now let us combine everything:

Employee.java

```
package com.nit.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import lombok.Data;

@Entity
@Table(name = "employee_details")
@Data
public class Employee {

    @Id
    private int id;

    private String name;
    private String address;
    private String companyName;
}
```

2.7 Java Object to Database Record

Suppose we create an Employee object:

```
Employee emp = new Employee();

emp.setId(101);
emp.setName("Prashant");
emp.setAddress("Hyderabad");
emp.setCompanyName("ABC");
```

Conceptually, the object contains:

```
id -> 101
name -> Prashant
address -> Hyderabad
companyName -> ABC
```

When Hibernate persists this object, the data can be stored in the database as a record:

employee_details

id	name	address	companyName
101	Prashant	Hyderabad	ABC

2.8 Java Class to Database Table Mapping

Java Entity	→	Database
Employee Class	→	employee_details
id	→	id
name	→	name
address	→	address
companyName	→	companyName
@Id on id	→	Primary Key

Now our class is ready to perform operations. Let us see how we can tell Hibernate to perform them.

In App.java

App.java

```
public class App
{
    public static void main( String[] args )
    {
        Configuration cnfig = new Configuration();
        cnfig.configure("resources/hibernate-cnf.xml");

        SessionFactory factory = cnfig.buildSessionFactory();
        System.out.println(factory);
        // Already we got the idea about the above part
        // The next part is:
    }
}
```

3. Hibernate Session and Transaction

After creating the Employee Entity, we have to save the Employee object into the database.

Reading employee details from the console

```
Employee employee = new Employee();
Scanner sc = new Scanner(System.in);
System.out.println("Enter the employee Id");
int id = sc.nextInt();
System.out.println("Enter the employee name");
String name = sc.next();
```

```
System.out.println("Enter the employee address");
String address = sc.next();
System.out.println("Enter the employee company name");
String company_Name = sc.next();

Employee emp = new Employee();
emp.setName(name);
emp.setId(id);
emp.setAddress(address);
emp.setCompanyName(company_Name);
```

For that, we use **Session** and **Transaction**. The basic flow is:



3.1 Open Session

```
Session session = factory.openSession();
```

SessionFactory is used to create a Session. Session is used to perform database operations with Hibernate, for example:

- Insert
- Update
- Delete
- Fetch

SIMPLE MEANING: openSession() → Open a Hibernate Session to work with the database.

3.2 Start Transaction

```
Transaction transaction = session.beginTransaction();
```

This starts a new transaction. A transaction is a unit of work. For example, if we want to insert an Employee, we perform that operation inside a transaction.

SIMPLE MEANING: beginTransaction() → Start the transaction.

3.3 Save the Employee

```
session.persist(emp);
```

Here, **emp** is our Employee object. Example:

```
Employee emp = new Employee();  
  
emp.setId(101);  
emp.setName("Prashant");  
  
session.persist(emp);
```

persist() tells Hibernate:

HIBERNATE: "I want to save this Employee object."

SIMPLE MEANING: persist(emp) → Tell Hibernate to save/manage this Entity object.

3.4 Commit the Transaction

```
transaction.commit();
```

After completing the operation, we commit the transaction. Commit means:

COMMIT: "Make the changes permanent."

SIMPLE MEANING: commit() → Make the transaction changes permanent.

3.5 Close the Session

```
session.close();
```

After completing the database work, we close the Session. This releases the resources used by the Session.

SIMPLE MEANING: close() → Close the Hibernate Session.

3.6 Complete Code

Full Session & Transaction flow

```
Session session = factory.openSession();
```

```
Transaction transaction = session.beginTransaction();  
  
session.persist(emp);  
  
transaction.commit();  
  
session.close();
```

End of Day 41 Notes — Hibernate (Day 02)